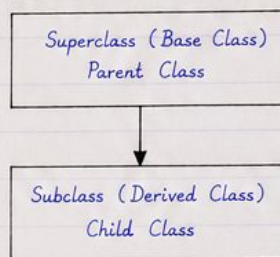


1. Inheritance (उत्तराधिकार) :

- Inheritance OOPs (Object Oriented Programming) का एक सबसे important concept है।
- इसके द्वारा एक class (subclass / derived class) दूसरी class (superclass / base class) के properties और behaviors को प्राप्त (inherit) करती है।
- यह code reusability बढ़ाने में मदद करता है और नए class बनाने को आसान बनाता है।
- Inheritance "is-a" relationship को represent करता है।

Inheritance का Diagram :



Syntax :

```

class SubClass extends SuperClass {
    // SubClass की body
}
  
```

Example :

```

class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
  
```

→ यहाँ Dog class, Animal class की properties और methods inherit करती है।

2. Access Modifiers in Inheritance :

Access Modifier	Same Class	Same Package	Subclass		Different Package (Non-subclass)	Meaning
			Same Package	Different Package		
public	Yes	Yes	Yes	Yes	Yes	• सबके द्वारा access किया जा सकता है। • Inheritance में भी accessible रहता है।
protected	Yes	Yes	Yes	Yes	No	• Same package और subclass (different package में) द्वारा access किया जा सकता है। • Non-subclass different package में नहीं।
(default)	Yes	Yes	Yes	No	No	• सिर्फ same package में access होता है। • Inheritance में different package में accessible नहीं होता।
private	Yes	No	No	No	No	• सिर्फ उसी class के अंदर access होता है। • Inheritance में कहीं भी accessible नहीं होता।

(A) Public Data (सार्वजनिक डेटा) :

- Public members को "public" keyword से declare किया जाता है।
- ये सभी classes, objects और packages के लिए accessible होते हैं।
- Inheritance में public members हमेशा derived class द्वारा access और reuse किए जा सकते हैं।

Example :

```

class Parent {
    public int x = 10;
    public void show() {
        System.out.println("Public data");
    }
}
  
```

→ Derived class में x और show() दोनों accessible होंगे।

(B) Protected Data (सुरक्षित डेटा) :

- Protected members को "protected" keyword से declare किया जाता है।
- ये same package की सभी classes और subclasses (चाहे वो अलग package में हों) द्वारा accessible होते हैं।
- Encapsulation और inheritance के बीच balance बनाता है।

Example :

```

class Parent {
    protected int y = 20;
    protected void display() {
        System.out.println("Protected data");
    }
}
  
```

→ Same package की classes और सभी subclasses इसे access कर सकती हैं।

(C) Default Data (पैकेज-निजी डेटा) :

- जब कोई access modifier specify नहीं किया जाता, तो वो default माना जाता है (package-private)।
- ये सिर्फ उसी package की classes के लिए accessible होते हैं।
- Inheritance में अगर subclass अलग package में है, तो default members access नहीं कर पाएगी।

Example :

```

class Parent {
    int z = 30; // default data
    void show() {
        System.out.println("Default data");
    }
}
  
```

→ Same package की classes access कर सकती हैं, अलग package में नहीं।

(D) Private Data (निजी डेटा) :

- Private members को "private" keyword से declare किया जाता है।
- ये सिर्फ उसी class के अंदर accessible होते हैं।
- ये inheritance में कहीं भी (subclass में भी) directly accessible नहीं होते।
- Encapsulation के लिए सबसे ज्यादा secure होता है।

Example :

```

class Parent {
    private int p = 40;
    private void secret() {
        System.out.println("Private data");
    }
}
  
```

→ p और secret() सिर्फ Parent class के अंदर ही accessible हैं, subclass में नहीं।

Note : ✓ Inheritance code reusability बढ़ाता है और program को easy to maintain बनाता है।
 ✓ Access modifiers का सही use data hiding (encapsulation) और security सुनिश्चन करता है।

1. Constructor Chaining (कंस्ट्रक्टर चेनिंग) :

- जब एक class का constructor दूसरे constructor को call करता है, तो इसे Constructor Chaining कहते हैं।
- यह chaining same class (using this()) या parent class (using super()) के constructor के बीच हो सकती है।
- इससे code reusability बढ़ती है और initialization आसान हो जाता है।
- Constructor chaining में circular call नहीं होना चाहिए।

(A) Constructor Chaining in Same Class (this()) :

```
class Student {
    int id; String name;
    Student() {
        this(0, "Unknown"); // current class के
    } // दूसरे constructor को call
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

(B) Constructor Chaining in Parent Class (super()) :

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}
class Student extends Person {
    int rollNo;
    Student(String name, int rollNo) {
        super(name); // Parent class का
        this.rollNo = rollNo; // constructor call
    }
}
```

Note :

- ✓ this() और super() constructor के first statement के रूप में होना चाहिए।
- ✓ this() same class के दूसरे constructor को call करता है।
- ✓ super() parent class के constructor को call करता है।

2. Order of Invocation (आह्वान का क्रम) :

- जब किसी derived class का object create होता है, तब constructors का call एक निश्चित क्रम में होता है।
- Invocation (call) हमेशा parent class से शुरू होकर child class तक जाता है।
- Instance variables और instance blocks (initialization blocks) का execution भी इसी क्रम में होता है।

Order of Invocation :

1. Super class (Base class) के static members और static blocks
2. Subclass (Derived class) के static members और static blocks
3. Super class के instance variables और instance blocks
4. Super class का constructor
5. Subclass के instance variables और instance blocks
6. Subclass का constructor

Example :

```
class A {
    static { System.out.println("A Static Block"); }
    int a = 10;
    { System.out.println("A Instance Block"); }
    A() { System.out.println("A Constructor"); }
}
class B extends A {
    static { System.out.println("B Static Block"); }
    int b = 20;
    { System.out.println("B Instance Block"); }
    B() { System.out.println("B Constructor"); }
}
class Test {
    public static void main(String[] args) {
        B obj = new B();
    }
}
```

Output (Call Order) :

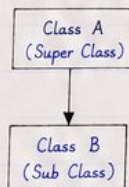
1. A Static Block
2. B Static Block
3. A Instance Block
4. A Constructor
5. B Instance Block
6. B Constructor

3. Types of Inheritance (उत्तराधिकार के प्रकार) :

→ Java में classes के बीच inheritance निम्न प्रकार के होते हैं :

(A) Single Inheritance

एक child class, एक parent class से inherit करती है।

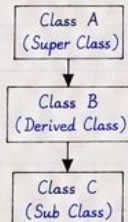


Example :

```
class A { }
class B extends A { }
```

(B) Multilevel Inheritance

एक child class, एक derived class से inherit करती है।

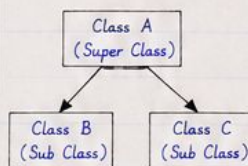


Example :

```
class A { }
class B extends A { }
class C extends B { }
```

(C) Hierarchical Inheritance

एक से अधिक child class, एक ही parent class से inherit करती है।

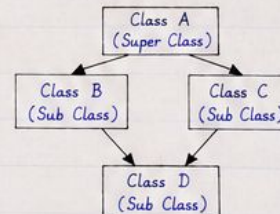


Example :

```
class A { }
class B extends A { }
class C extends A { }
```

(D) Hybrid Inheritance

उपर के दो या दो से अधिक प्रकार का combination.



Example :

```
class A { }
class B extends A { }
class C extends A { }
class D extends B { }
// (also C can extend B)
```

Important Points :

- ✓ Java में Multiple Inheritance (एक से अधिक parent class से inherit) allowed नहीं है (क्योंकि ambiguity की समस्या होती है)।
- ✓ Interfaces के माध्यम से Multiple Inheritance achieve की जा सकती है।
- ✓ Inheritance code reusability, method overriding और extensibility के लिए बहुत useful है।

1. Single Inheritance (एकल उत्तराधिकार):

- जब एक child class केवल एक ही parent class से inherit करती है, उसे Single Inheritance कहते हैं।
- इसमें child class को parent class के सभी public और protected members प्राप्त होते हैं।
- Java में class केवल एक ही class को extend कर सकती है, इसलिए यह सबसे common type of inheritance है।

Example :

```
class Person {
    String name;
    int age;
    void showPerson() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
class Student extends Person {
    int rollNo;
    void showStudent() {
        System.out.println("Roll No: " + rollNo);
    }
}
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Amit";           // inherited from Person
        s.age = 20;                // inherited from Person
        s.rollNo = 101;           // own member
        s.showPerson();           // parent class method
        s.showStudent();          // child class method
    }
}
```

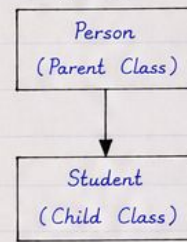
Explanation :

- Student class, Person class को extend कर रही है।
- Student को Person के सभी public और protected members (name, age, showPerson()) प्राप्त होते हैं।
- Student अपनी खुद की property (rollNo) और method (showStudent()) भी रखती है।
- main() में Student object के द्वारा parent class की method भी call की जा सकती है।

Key Points :

1.	Relationship	IS-A
2.	Parent Class	एक
3.	Child Class	एक
4.	Extend Keyword	extends
5.	Example	Person → Student

Diagram :



2. Multilevel Inheritance (बहुस्तरीय उत्तराधिकार):

- जब एक class दूसरी class को extend करती है, और वह derived class आगे किसी और class को extend करती है, तो इसे Multilevel Inheritance कहते हैं।
- इसमें inheritance कई levels तक होता है।
- यह भी IS-A relationship को represent करता है।

Example :

```
class Grandparent {
    void showGP() {
        System.out.println("I am Grandparent");
    }
}
class Parent extends Grandparent {
    void showParent() {
        System.out.println("I am Parent");
    }
}
class Child extends Parent {
    void showChild() {
        System.out.println("I am Child");
    }
}
class Test {
    public static void main(String[] args) {
        Child c = new Child();
        c.showGP();           // from Grandparent
        c.showParent();       // from Parent
        c.showChild();        // from Child
    }
}
```

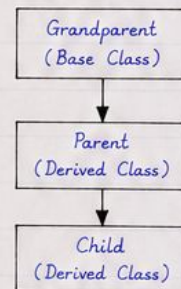
Explanation :

- Child class, Parent class को extend करती है, और Parent class, Grandparent class को extend करती है।
- इस प्रकार Child class को Grandparent और Parent दोनों के सभी accessible members प्राप्त होते हैं।
- main() में Child object के द्वारा तीनों class की methods call की जा सकती हैं।

Key Points :

1.	Relationship	IS-A (chain form)
2.	Levels	Multi (2 या अधिक)
3.	Inheritance Flow	Grandparent → Parent → Child
4.	Extend Keyword	हर level पर extends
5.	Example	Grandparent → Parent → Child

Diagram :



Important Note :

- ✓ Java में class केवल ही class को extend कर सकती है, इसलिए Multiple Inheritance of class allowed नहीं है।
- ✓ लेकिन Multilevel Inheritance पूरी तरह से allowed है।
- ✓ Object बनाने पर सबसे पहले top-most class (Grandparent) का constructor call होता है।

1. Hierarchical Inheritance (पदानुक्रमीय उत्तराधिकार):

→ जब एक parent class से एक से अधिक child classes inherit करती हैं, परन्तु child classes आपस में related नहीं होतीं, तो इसे Hierarchical Inheritance कहते हैं।

→ इसमें सभी child classes को parent class के सभी accessible members प्राप्त होते हैं।

→ यह IS-A relationship को represent करता है।

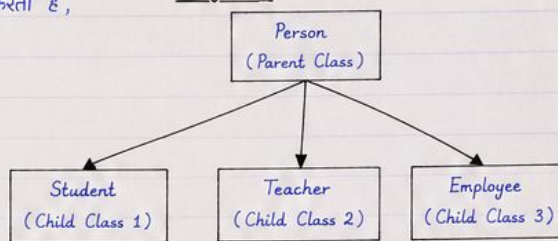
Example:

```
class Person {
    String name;
    int age;
    void showPerson() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
class Student extends Person {
    int rollNo;
    void showStudent() {
        System.out.println("Roll No: " + rollNo);
    }
}
class Teacher extends Person {
    String subject;
    void showTeacher() {
        System.out.println("Subject: " + subject);
    }
}
class Employee extends Person {
    double salary;
    void showEmployee() {
        System.out.println("Salary: " + salary);
    }
}
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Amit"; s.age = 20; s.rollNo = 101;
        s.showPerson(); s.showStudent();

        Teacher t = new Teacher();
        t.name = "Ravi"; t.age = 30; t.subject = "Maths";
        t.showPerson(); t.showTeacher();

        Employee e = new Employee();
        e.name = "Rahul"; e.age = 25; e.salary = 25000;
        e.showPerson(); e.showEmployee();
    }
}
```

Diagram:



Explanation:

- Person class parent है और Student, Teacher, Employee उसके child classes हैं।
- सभी child classes को Person के name और age members मिलते हैं।
- सभी child classes अपनी-अपनी विशेषताएं (rollNo, subject, salary) और methods define करती हैं।
- main() में प्रत्येक child class का object बनाकर methods call की जाती हैं।

Key Points:

1.	Parent Class	एक
2.	Child Classes	एक से अधिक
3.	Relationship	IS-A
4.	Child classes आपस में related नहीं होतीं	
5.	Code Reusability	अधिक
6.	Example	Person → Student, Teacher, Employee

2. Hybrid Inheritance (संकर उत्तराधिकार):

→ जब एक से अधिक प्रकार की inheritance (जैसे Hierarchical + Multilevel) एक साथ use होती हैं, तो इसे Hybrid Inheritance कहते हैं।

→ इसमें एक parent class से कई child classes होती हैं और उन child classes में से किसी एक से further child class बनती हैं।

→ यह inheritance अधिक flexible और real-world scenarios के लिए useful होती है।

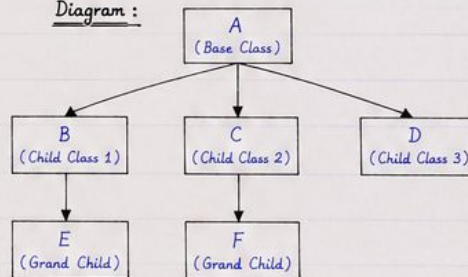
Example:

```
class A {
    void showA() { System.out.println("A class method"); }
}
class B extends A {
    void showB() { System.out.println("B class method"); }
}
class C extends A {
    void showC() { System.out.println("C class method"); }
}
class D extends A {
    void showD() { System.out.println("D class method"); }
}
class E extends B {
    void showE() { System.out.println("E class method"); }
}
class F extends C {
    void showF() { System.out.println("F class method"); }
}
class Test {
    public static void main(String[] args) {
        E obj1 = new E();
        obj1.showA(); obj1.showB(); obj1.showE();

        F obj2 = new F();
        obj2.showA(); obj2.showC(); obj2.showF();

        D obj3 = new D();
        obj3.showA(); obj3.showD();
    }
}
```

Diagram:



Explanation:

- A base class है, B, C और D उसकी child classes हैं (Hierarchical).
- B और C से क्रमशः E और F inherit करती हैं (Multilevel).
- इस प्रकार यह combination Hybrid Inheritance कहलाता है।

Key Points:

1.	Multiple inheritance types का combination होता है
2.	Code reuse अधिक होता है
3.	Hierarchical + Multilevel most common combination है
4.	Real-world applications के लिए उपयोगी
5.	Example A → (B, C, D), B → E, C → F

Important Note:

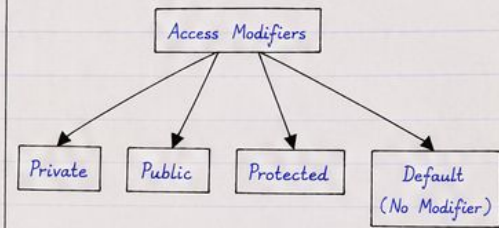
- ✓ Java में Multiple Inheritance of Classes allowed नहीं है।
- ✓ परन्तु Interfaces के माध्यम से Multiple Inheritance achieve की जा सकती है।
- ✓ Hybrid Inheritance design को अधिक modular और maintainable बनाती है।

1. Access Control (एक्सेस कंट्रोल):

- Access Control Java में access modifiers के द्वारा किया जाता है।
- ये modifiers class के members (variables, methods, constructors) की visibility को define करते हैं कि उन्हें कहाँ access किया जा सकता है।
- Java में चार प्रकार के access modifiers होते हैं:

1. Private
2. Public
3. Protected
4. Default (No Modifier)

Concept Diagram:



(A) Access Modifier का Scope (कहाँ-कहाँ से access किया जा सकता है):

Access Modifier	Same Class	Same Package (Same Class)	SubClass in Same Package	SubClass in Different Package	Non-Subclass in Different Package
Private	Yes	No	No	No	No
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default (No Modifier)	Yes	Yes	No	No	No

(1) Private :

- Private members केवल उसी class के अंदर access होते हैं।
- इन्हें class के बाहर या subclass से access नहीं किया जा सकता है।
- Encapsulation के लिए सबसे ज्यादा secure modifier है।
- Private data को direct access नहीं किया जा सकता, उसके लिए getter-setter methods का use किया जाता है।

Example :

```

class A {
    private int x = 10;
    private void show() {
        System.out.println("Private method");
    }
}
  
```

→ केवल class A के अंदर access होगा।

(2) Public :

- Public members सभी जगह से access किए जा सकते हैं।
- Same class, same package, subclass (same/diff package) और non-subclass (diff package) सभी access कर सकते हैं।
- Public members का use class के बाहर freely किया जा सकता है।

Example :

```

class A {
    public int x = 10;
    public void show() {
        System.out.println("Public method");
    }
}
  
```

→ सभी जगह से accessible

(3) Protected :

- Protected members उसी package की सभी classes और subclasses (same या different package) में access होते हैं।
- Non-subclass class (different package) से access नहीं होते हैं।
- Inheritance में use के लिए बहुत useful होता है।

Example :

```

class A {
    protected int x = 10;
    protected void show() {
        System.out.println("Protected method");
    }
}
  
```

→ Same package और subclass (सभी packages) से access होगा।

(4) Default (No Modifier):

- जब हम कोई modifier नहीं लिखते, तो वह Default (package-private) होता है।
- यह members केवल उसी package की classes से access होते हैं।
- Different package की classes और subclasses से access नहीं होते हैं।

Example :

```

class A {
    int x = 10; // default
    void show() {
        System.out.println("Default method");
    }
}
  
```

→ केवल same package की classes से access होगा।

Important Points :

- ✓ Private < Default < Protected < Public (visibility के increasing order)
- ✓ Private modifier security और encapsulation के लिए use होता है।
- ✓ Public modifier API design और wide access के लिए use होता है।
- ✓ Protected modifier inheritance के द्वारा reuse को support करता है।
- ✓ Default modifier package level access के लिए use होता है।

Note :

- Access Modifiers का सही use program को secure, maintainable और reusable बनाता है।